

Object Oriented Programming – CS/CE 224/272

Nadia Nasir



Content



Dangling pointers

Null Pointers (nullptr)

Understand dangling pointers by using example

```
#include<iostream>
```

```
int main()  
{  
    int x{ 5 };  
    int* ptr{ &x };  
  
    if (true)  
    {  
        int y{ 6 };  
        ptr = &y;  
    }  
  
    std::cout << *ptr << '\n';  
  
    return 0;  
}
```

int x

5

0x10

```
#include<iostream>
```

```
int main()
```

```
{  
  int x{ 5 };  
  int* ptr{ &x };  
  
```

```
  if (true)
```

```
  {  
    int y{ 6 };  
    ptr = &y;  
  }
```

```
  std::cout << *ptr << '\n';
```

```
  return 0;  
}
```

int* ptr

0x10

0x40

int x

5

0x10



```
#include<iostream>
```

```
int main()
{
    int x{ 5 };
    int* ptr{ &x };

    if (true)
    {
        int y{ 6 };
        ptr = &y;
    }

    std::cout << *ptr << '\n';

    return 0;
}
```

int* ptr

0x10

0x40

int x

5

0x10

int y

6

0x18



```
#include<iostream>
```

```
int main()
```

```
{
```

```
    int x{ 5 };
```

```
    int* ptr{ &x };
```

```
    if (true)
```

```
    {
```

```
        int y{ 6 };
```

```
        ptr = &y;
```

```
    }
```

```
    std::cout << *ptr << '\n';
```

```
    return 0;
```

```
}
```

int* ptr

0x18

0x40

int x

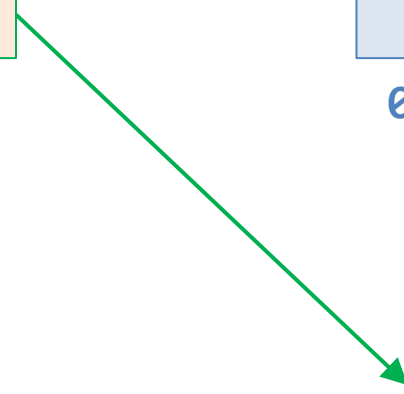
5

0x10

int y

6

0x18



```
#include<iostream>
```

```
int main()
```

```
{  
    int x{ 5 };  
    int* ptr{ &x };
```

```
    if (true)
```

```
    {  
        int y{ 6 };  
        ptr = &y;  
    }
```

```
    std::cout << *ptr << '\n';
```

```
    return 0;
```

```
}
```

int* ptr

0x18

0x40

int x

5

0x10

int y

6

0x18

What will be the
output of this
line?

Terminal> ???

```
#include<iostream>
```

```
int main()
```

```
{  
    int x{ 5 };  
    int* ptr{ &x };
```

```
    if (true)
```

```
    {  
        int y{ 6 };  
        ptr = &y;
```

```
    }  
    std::cout << *ptr << '\n';
```

```
    return 0;
```

```
}
```

int* ptr

0x18

0x40

int x

5

0x10

int y

6

0x18

Variable y goes
out of scope
here...

Terminal> ???

```
#include<iostream>
```

```
int main()
```

```
{
```

```
    int x{ 5 };
```

```
    int* ptr{ &x };
```

```
    if (true)
```

```
    {
```

```
        int y{ 6 };
```

```
        ptr = &y;
```

```
    }
```

```
    std::cout << *ptr << '\n';
```

```
    return 0;
```

```
}
```

int* ptr

0x18

0x40

int x

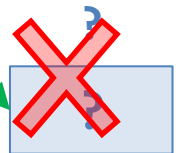
5

0x10

We don't know what is
in this memory address
now.

It's no longer in our
possession.

Terminal> ???



```
#include<iostream>
```

```
int main()
{
    int x{ 5 };
    int* ptr{ &x };

```

```
    if (true)
    {
        int y{ 6 };
        ptr = &y;
    }

```

```
    std::cout << *ptr << '\n';

```

```
    return 0;

```

int* ptr

0x18

0x40

int x

5

0x10

Pointer, **ptr**, is left
DANGLING, i.e. it's
not pointing to a
valid memory
address.

Terminal> ???

0x18

```
#include<iostream>
```

```
int main()
```

```
{  
    int x{ 5 };  
    int* ptr{ &x };
```

```
    if (true)  
    {  
        int y{ 6 };  
        ptr = &y;
```

```
    }  
    std::cout << *ptr << '\n';
```

```
    return 0;
```

```
}
```

int* ptr

0x18

0x40

int x

5

0x10

?

0x18

You get **undefined behavior** if you try to dereference it.

Terminal> UNDEFINED BEHAVIOR

In some cases, it may actually print value of y, but may lead to an error as well.

Runtime error (not compile time).

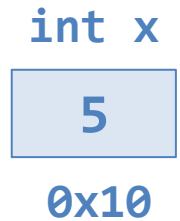
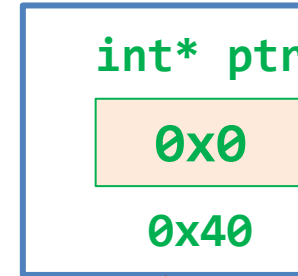
```
#include<iostream>
```

```
int main()
{
    int x{ 5 };
    int* ptr{ &x };

    if (true)
    {
        int y{ 6 };
        ptr = &y;
    }

    ptr = nullptr;

    return 0;
}
```



Set dangling
pointers to null,
i.e. they are
pointing
nowhere.

Terminal> Nice!

```
#include<iostream>
```

```
int main()
{
    int x{ 5 };
    int* ptr{ &x };

    if (true)
    {
        int y{ 6 };
        ptr = &y;
    }

    ptr = nullptr;

    return 0;
}
```

Can you think of a possible sequel to this story?

The story actually hasn't ended yet....



```
#include<iostream>
```

```
int main()
{
    int x{ 5 };
    int* ptr{ &x };

    if (true)
    {
        int y{ 6 };
        ptr = &y;
    }

    ptr = nullptr;

    std::cout << *ptr << '\n';

    return 0;
}
```

int* ptr

0x0

0x40

int x

5

0x10



0x18

What will happen on
dereferencing **ptr**
while it's pointing
nowhere?

Terminal> ???

```
#include<iostream>
```

```
int main()
{
    int x{ 5 };
    int* ptr{ &x };

    if (true)
    {
        int y{ 6 };
        ptr = &y;
    }

    ptr = nullptr;

    std::cout << *ptr << '\n';

    return 0;
}
```

int* ptr

0x0

0x40

int x

5

0x10



0x18

This will also be
**UNDEFINED
BEHAVIOR.**

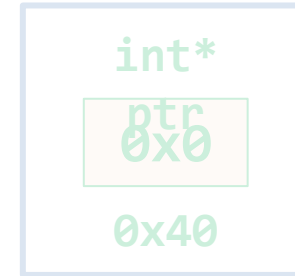
Terminal> UNDEFINED BEHAVIOR
*(Mostly likely a program
crash).*

```
#include<iostream>
```

```
int main()
{
    int x{ 5 };
    int* ptr{ &x };

    if (true)
    {
        int y{ 6 };
        ptr = &y;
    }

    ptr = nullptr;
    std::cout << *ptr << endl;
    return 0;
}
```



*Any
workarounds?*

Terminal> UNDEFINED BEHAVIOR
(Mostly likely a program
crash).

```
#include<iostream>
```

```
int main()
{
    int x{ 5 };
    int* ptr{ &x };

```

```
    if (true)
    {
        int y{ 6 };
        ptr = &y;
    }

```

```
    ptr = nullptr;

```

```
    if (ptr == nullptr)
        std::cout << "ptr is null\n";
    else
        std::cout << *ptr << '\n';

```

```
    return 0;
}
```

int* ptr

0x0

0x40

int x

5

0x10



0x18

Check for null
pointers...

Terminal> ptr is null

// Program didn't crash...

```
#include<iostream>
```

```
int main()
{
    int x{ 5 };
    int* ptr{ &x };

```

```
    if (true)
    {
        int y{ 6 };
        ptr = &y;
    }

```

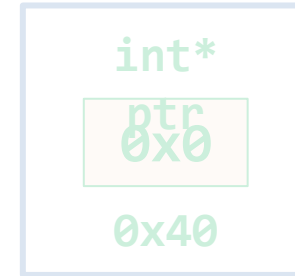
```
    ptr = nullptr;

```

```
    if (ptr == nullptr)
        std::cout << "ptr is null\n";
    else
        std::cout << *ptr << '\n';

```

```
    return 0;
}
```



The End

Check for null pointers...

```
Terminal> ptr is null
```

```
// Program didn't crash...
```

Question (Self Study)

Can I have somehow not set the pointer to **nullptr** and just used a conditional to dereference only when my pointer is not dangling?

Answer is discussed in the end of this slide deck and in the provided link in the self study material.



Dangling Pointers

Your pointers should point to valid entities.

Or they should be set to null.

So now, only **check for null pointers** before working with them...

... And any **non-null pointers** can be assume to be **valid**.

Some Situations to be Mindful of...

Some situations where pointers may be left dangling.

Returning pointers from functions.

If they're pointing to local variables, those local variables have gone out of scope (can you craft a program in which this happens?).

Something similar to the example discussed in this lecture.

When working with dynamic memory (coming soon...).

Dangling Pointers

The code used in this example is also available in the relevant code file.

6-pointers_dangling.cpp

I have used an **if** block in the slides, but you can just create a regular code block as well.



Null Pointers (nullptr)

Note that in our discussion thus far, we have also talked about null pointers.

Its definition (what does it mean).

What happens when a null pointer is dereferenced, and why does it happen?

Use them to avoid wild pointers

```
int* p;          // uninitialized
```

Use them to avoid dangling pointers.

Checking for null pointers using conditionals.

Self-Reading



Following slides contain some useful screenshots relevant to the topic of **null pointers** and **dangling pointers**.

These screenshots have been taken from the following link.

<https://www.learncpp.com/cpp-tutorial/null-pointers/>

The following program illustrates this, and will probably crash or terminate your application abnormally when you run it (go ahead, try it, you won't harm your machine):

```
1 | #include <iostream>
2 |
3 | int main()
4 | {
5 |     int* ptr {}; // Create a null pointer
6 |     std::cout << *ptr << '\n'; // Dereference the null pointer
7 |
8 |     return 0;
9 | }
```

Explanation about why dereferencing null pointers doesn't make any sense.

Conceptually, this makes sense. Dereferencing a pointer means “go to the address the pointer is pointing at and access the value there”. A null pointer holds a null value, which semantically means the pointer is not pointing at anything. So what value would it access?

Accidentally dereferencing null and dangling pointers is one of the most common mistakes C++ programmers make, and is probably the most common reason that C++ programs crash in practice.

Warning

Whenever you are using pointers, you'll need to be extra careful that your code isn't dereferencing null or dangling pointers, as this will cause undefined behavior (probably an application crash).

Warning

Conditionals can only be used to differentiate null pointers from non-null pointers. There is no convenient way to determine whether a non-null pointer is pointing to a valid object or dangling (pointing to an invalid object).

We can easily avoid dereferencing a null pointer by using a conditional to ensure a pointer is non-null before trying to dereference it:

```
1 // Assume ptr is some pointer that may or may not be a null pointer
2 if (ptr) // if ptr is not a null pointer
3     std::cout << *ptr << '\n'; // okay to dereference
4 else
5     // do something else that doesn't involve dereferencing ptr (print an error message,
    // do nothing at all, etc...)
```

But what about dangling pointers? Because there is no way to detect whether a pointer is dangling, we need to avoid having any dangling pointers in our program in the first place. We do that by ensuring that any pointer that is not pointing at a valid object is set to `nullptr`.

That way, before dereferencing a pointer, we only need to test whether it is null -- if it is non-null, we assume the pointer is not dangling.

Best practice

A pointer should either hold the address of a valid object, or be set to `nullptr`. That way we only need to test pointers for null, and can assume any non-null pointer is valid.